

Exploratory Thoughts: Unified Recommender System Architecture

Transformer Sequence Modeling + Feature Interaction

algo.qq.com

David Xia

April 2026

Draft v0.1

Prepared in the context of the QQ Algorithm Challenge — algo.qq.com

Abstract

These are first-thoughts towards a unified recommender system architecture that merges transformer-based sequence modelling with structured feature interaction, targeting the QQ Algorithm Challenge. The intent is exploratory: to sketch the mathematical and implementation foundations before committing to a full proposal.

Keywords: recommender systems, transformers, feature interaction, PyTorch, deep learning

1 Overview

This document presents a compact technical model describing how a unified recommender-system architecture can merge:

- Transformer-style sequence modeling,
- Feature-interaction modeling (e.g. DeepFM, DCN, DIN),
- A unified PyTorch implementation,
- A training pipeline and framework comparison.

The goal is to show how modern recommender systems integrate temporal dependencies and structured feature interactions into a single scalable model.

2 1. Minimal Unified Architecture (PyTorch)

Below is a minimal PyTorch model that combines:

- a Transformer encoder for sequential user behaviour,
- static feature embeddings (user, item, context),
- an MLP for feature interactions.

Model Code

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class UnifiedRecModel(nn.Module):
    def __init__(
        self,
        num_items,
        num_users,
        num_context_feats,
        d_model=64,
        n_heads=4,
        d_ff=128,
        seq_max_len=50,
        mlp_hidden=128,
    ):
        super().__init__()

        self.item_emb = nn.Embedding(num_items, d_model)
        self.user_emb = nn.Embedding(num_users, d_model)
        self.ctx_emb = nn.Embedding(num_context_feats, d_model)

        encoder_layer = nn.TransformerEncoderLayer(
            d_model=d_model,
            nhead=n_heads,
            dim_feedforward=d_ff,
            batch_first=True,
        )
        self.seq_encoder = nn.TransformerEncoder(encoder_layer, num_layers=2)

        in_dim = d_model * 4
        self.mlp = nn.Sequential(
            nn.Linear(in_dim, mlp_hidden),
            nn.ReLU(),
```

```

        nn.Linear(mlp_hidden, 1),
    )

def forward(self, seq_items, user_ids, target_items, ctx_feats, seq_mask=None):
    seq_emb = self.item_emb(seq_items)

    if seq_mask is not None:
        src_key_padding_mask = (seq_mask == 0)
    else:
        src_key_padding_mask = None

    seq_encoded = self.seq_encoder(
        seq_emb,
        src_key_padding_mask=src_key_padding_mask
    )

    if seq_mask is not None:
        lengths = seq_mask.sum(dim=1, keepdim=True).clamp(min=1)
        seq_pooled = (seq_encoded * seq_mask.unsqueeze(-1)).sum(dim=1) / lengths
    else:
        seq_pooled = seq_encoded.mean(dim=1)

    user_vec = self.user_emb(user_ids)
    item_vec = self.item_emb(target_items)
    ctx_vec = self.ctx_emb(ctx_feats).mean(dim=1)

    x = torch.cat([seq_pooled, user_vec, item_vec, ctx_vec], dim=-1)
    logit = self.mlp(x).squeeze(-1)
    return logit

```

3 2. Merging Attention and Feature-Interaction Layers

A unified architecture combines:

- Transformer attention for temporal modelling,
- Feature-interaction layers (e.g. FM, DeepFM) for structured fields.

Attention

$$\text{Attn}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V.$$

Factorization Machine Interaction

$$y_{\text{FM}} = \frac{1}{2} \sum_{i < j} (\langle v_i, v_j \rangle).$$

FM Layer Code

```
class FMInteraction(nn.Module):
    def __init__(self, dim):
        super().__init__()
        self.dim = dim

    def forward(self, field_embs):
        sum_emb = field_embs.sum(dim=1)
        sum_sq = sum_emb * sum_emb
        sq_sum = (field_embs * field_embs).sum(dim=1)
        fm = 0.5 * (sum_sq - sq_sum).sum(dim=1)
        return fm
```

4 3. Training Pipeline

A typical training pipeline includes:

1. Data preparation (sequence padding, masks, feature IDs).
2. PyTorch Dataset and DataLoader.
3. Training loop with BCE loss.
4. Validation using AUC and log-loss.

Training Loop

```
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
criterion = nn.BCEWithLogitsLoss()

for epoch in range(num_epochs):
    model.train()
    for batch in train_loader:
        logits = model(
            batch["seq_items"].to(device),
            batch["user_ids"].to(device),
            batch["target_items"].to(device),
            batch["ctx_feats"].to(device),
```

```
        batch["seq_mask"].to(device),
    )
    loss = criterion(logits, batch["labels"].float().to(device))
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
```

5 4. PyTorch vs. TensorFlow

- **PyTorch**: flexible, research-friendly, strong Transformer ecosystem.
- **TensorFlow**: strong for production CTR/CVR models, TF-Serving.
- **Deployment**: ONNX, TorchScript, TensorRT, TF-Serving.
- **Trend**: PyTorch dominates research; TF remains strong in industry.